

Week13HW-checkpoint

June 8, 2025

1 Week 13 Assignment

For both of these games, follow the process outlined in the video to break them down into smaller component functions. Add a markdown cell to the notebook to explain your process and the options you considered for each one.

1.1 Problem 1

In the cell(s) below, create a number guessing game, where python chooses a random integer, and tells the player whether their guess is high, low, or correct.

```
[ ]: # Import needed for the computer to select the random number
import random

def number_guessing_game():
    number = random.randint(1, 100) # Computer picks a number between 1 and 100
    print("I'm thinking of a number between 1 and 100. Try to guess it!")

    while True:
        guess = input("Enter your guess: ")
        if not guess.isdigit():
            print("Please enter a valid number.")
            continue
        guess = int(guess)

        if guess < number:
            print("Too low! Try again.")
        elif guess > number:
            print("Too high! Try again.")
        else:
            print("Correct! You guessed the number!")
            break

if __name__ == "__main__":
    number_guessing_game()
```

```
I'm thinking of a number between 1 and 100. Try to guess it!
Too low! Try again.
Too high! Try again.
```

Too low! Try again.
Too low! Try again.
Correct! You guessed the number!

1.2 Problem 2

In the cell(s) below, create a tic-tac-toe game to play against the computer.

```
[1]: import random

def print_board(board):
    for row in board:
        print(" ".join(row))

def player_move(board):
    while True:
        move = input("Enter your move (1-9): ")
        if move.isdigit():
            pos = int(move) - 1
            row, col = divmod(pos, 3)
            if 0 <= pos <= 8 and board[row][col] == "-":
                board[row][col] = "X"
                break
            else:
                print("Invalid move, spot taken or out of range.")
        else:
            print("Please enter a number from 1 to 9.")

def computer_move(board):
    empty_positions = [(r, c) for r in range(3) for c in range(3) if
        ↪board[r][c] == "-"]
    if empty_positions:
        row, col = random.choice(empty_positions)
        board[row][col] = "O"
        print(f"Computer chose position {row * 3 + col + 1}")

def check_winner(board, player):
    # rows, cols, diagonals
    for i in range(3):
        if all(board[i][j] == player for j in range(3)):
            return True
        if all(board[j][i] == player for j in range(3)):
            return True
    if all(board[i][i] == player for i in range(3)):
        return True
    if all(board[i][2 - i] == player for i in range(3)):
        return True
    return False
```

```

def is_board_full(board):
    return all(cell != "-" for row in board for cell in row)

def tic_tac_toe():
    board = [["-" for _ in range(3)] for _ in range(3)]
    print("Positions:\n1 2 3\n4 5 6\n7 8 9\n")
    while True:
        print_board(board)
        player_move(board)
        if check_winner(board, "X"):
            print_board(board)
            print("You win!")
            break
        if is_board_full(board):
            print_board(board)
            print("It's a tie!")
            break

        computer_move(board)
        if check_winner(board, "O"):
            print_board(board)
            print("Computer wins!")
            break
        if is_board_full(board):
            print_board(board)
            print("It's a tie!")
            break

if __name__ == "__main__":
    tic_tac_toe()

```

Positions:

```

1 2 3
4 5 6
7 8 9

```

```

- - -
- - -
- - -

```

Computer chose position 1

```

0 - -
X - -
- - -

```

Computer chose position 3

```

0 - 0
X X -
- - -

```

```
0 - 0
X X X
- - -
You win!
```

1.3 Problem 3

The code in the cells below generates errors. Describe what is causing the error and change the code so that the cells evaluate correctly.

```
[ ]: # For these first 2 lines we corrected it to take in any arguement
def just_say_no(message):
    print(message)

# The argument here is.. 'No!'
just_say_no('No!')
```

No!

```
[ ]: # We needed to use paranthesis instead of brackets
def addition(a, b):
    return a + b

# OPTIONAL: TESTING FUNCTION WORKS
result = addition(4, 6)
print(result) # This will print out the result
```

10

```
[ ]: def algebra(a,b,c):
    answer = a**2-2*b+c
    return answer

# Testing that the function works
result = algebra(12,14,19) # we needed to assign algebra to result in order to
    ↪ call it at the end
print(result) # calling to print out result to get our final result.
```

135